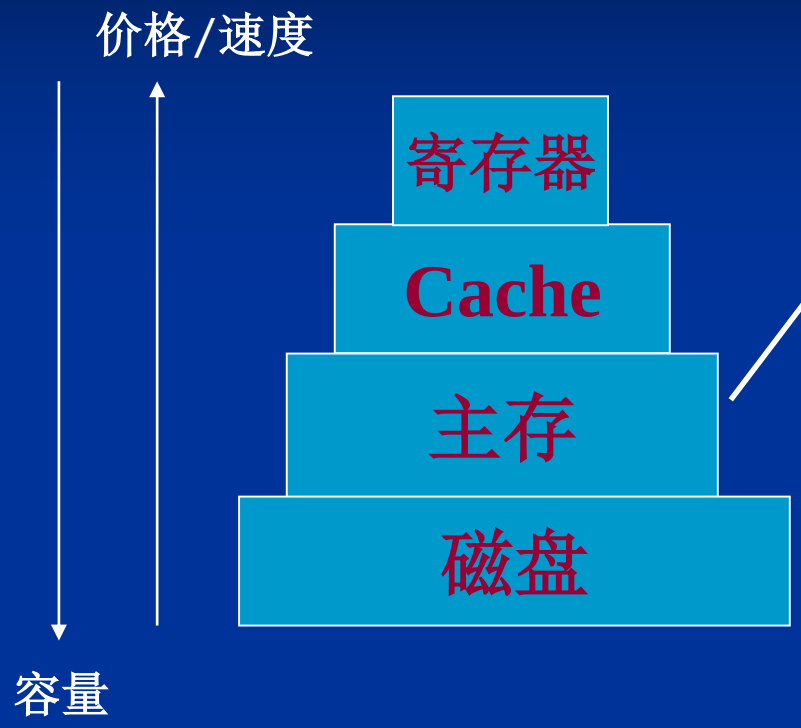
第4章 存储管理

存储体系概述

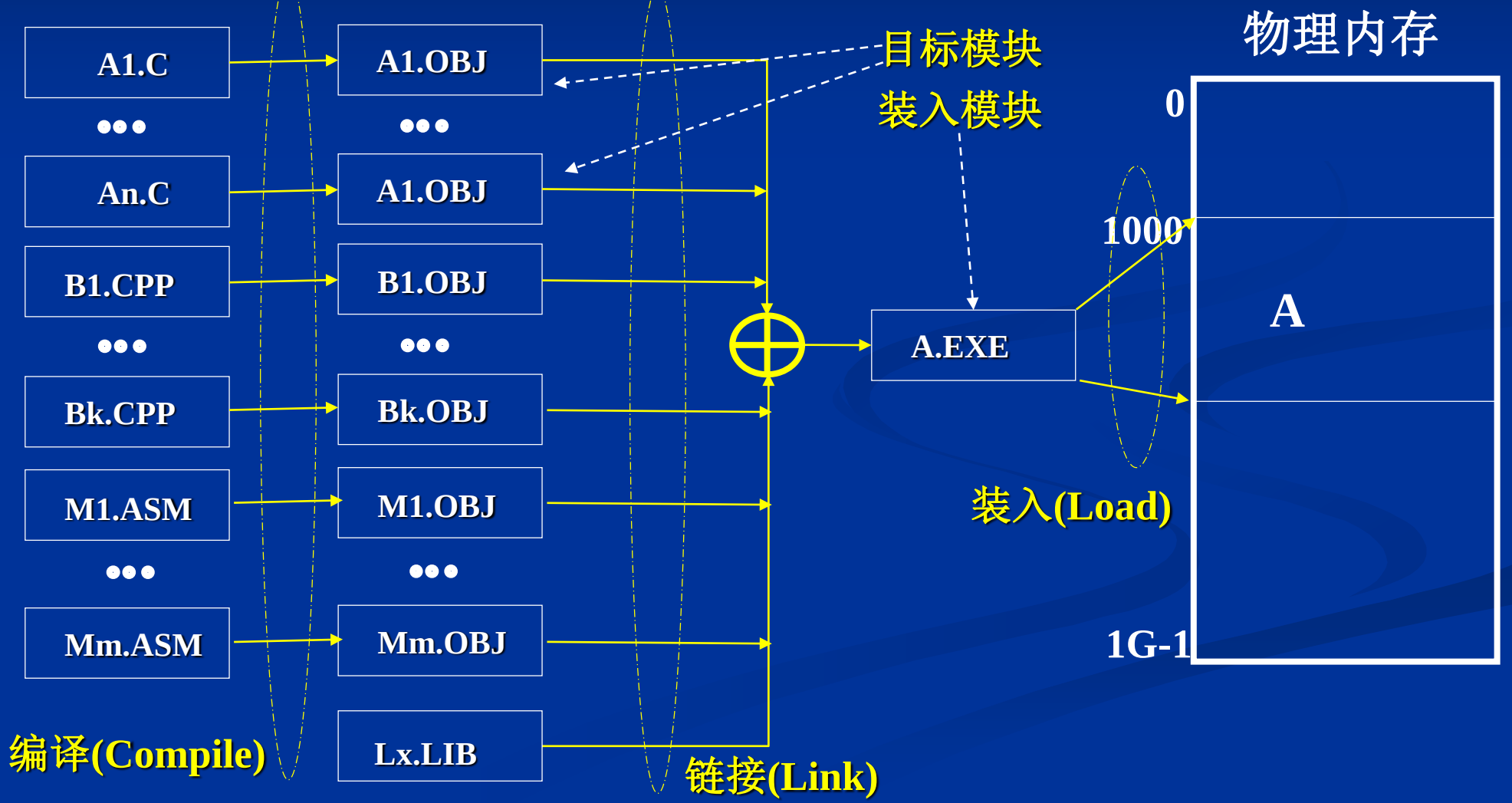


- 由存储单元（字节或字）组成的一维连续的地址空间，简称内存空间。
- 用来存放当前正在运行程序的代码及数据，是程序中指令本身地址所指的、亦即程序计数器所指的存储器
- 性能“瓶颈”：关键、紧张
- 帕金森定律：

  内存多大，程序多长

4.1 程序的装入和链接

4.1.0 程序的编程、编译、链接和装入



4.1 程序的装入和链接

4.1.1 程序的装入

- 绝对装入方式

  编程时，直接确定内存位置；

  特点：

  不灵活，不支持多道程序设计技术；

4.1 程序的装入和链接

4.1.1 程序的装入

- 可重定位装入方式

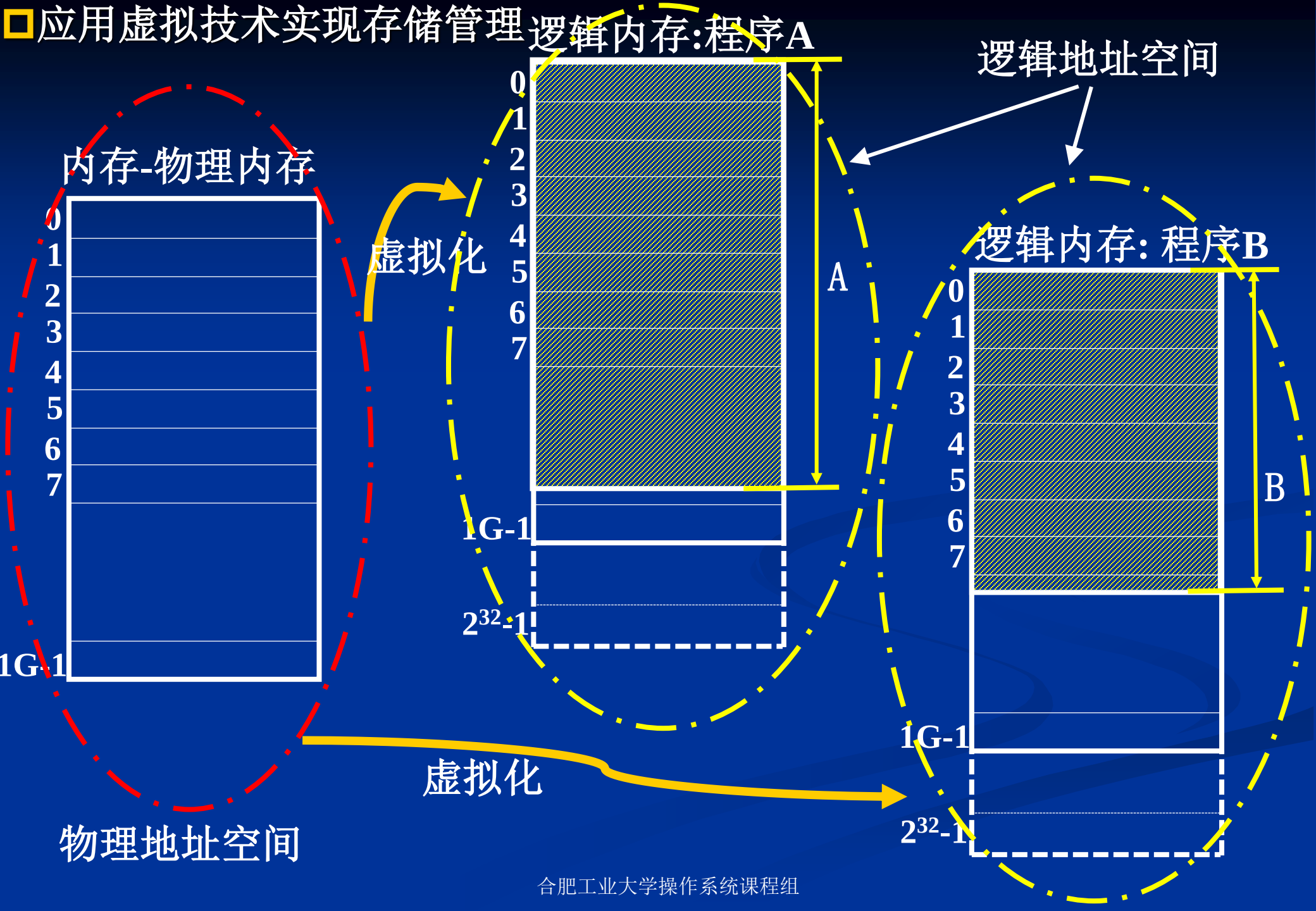
  虚拟技术：编程时使用逻辑地址，运行时使用物理地址；

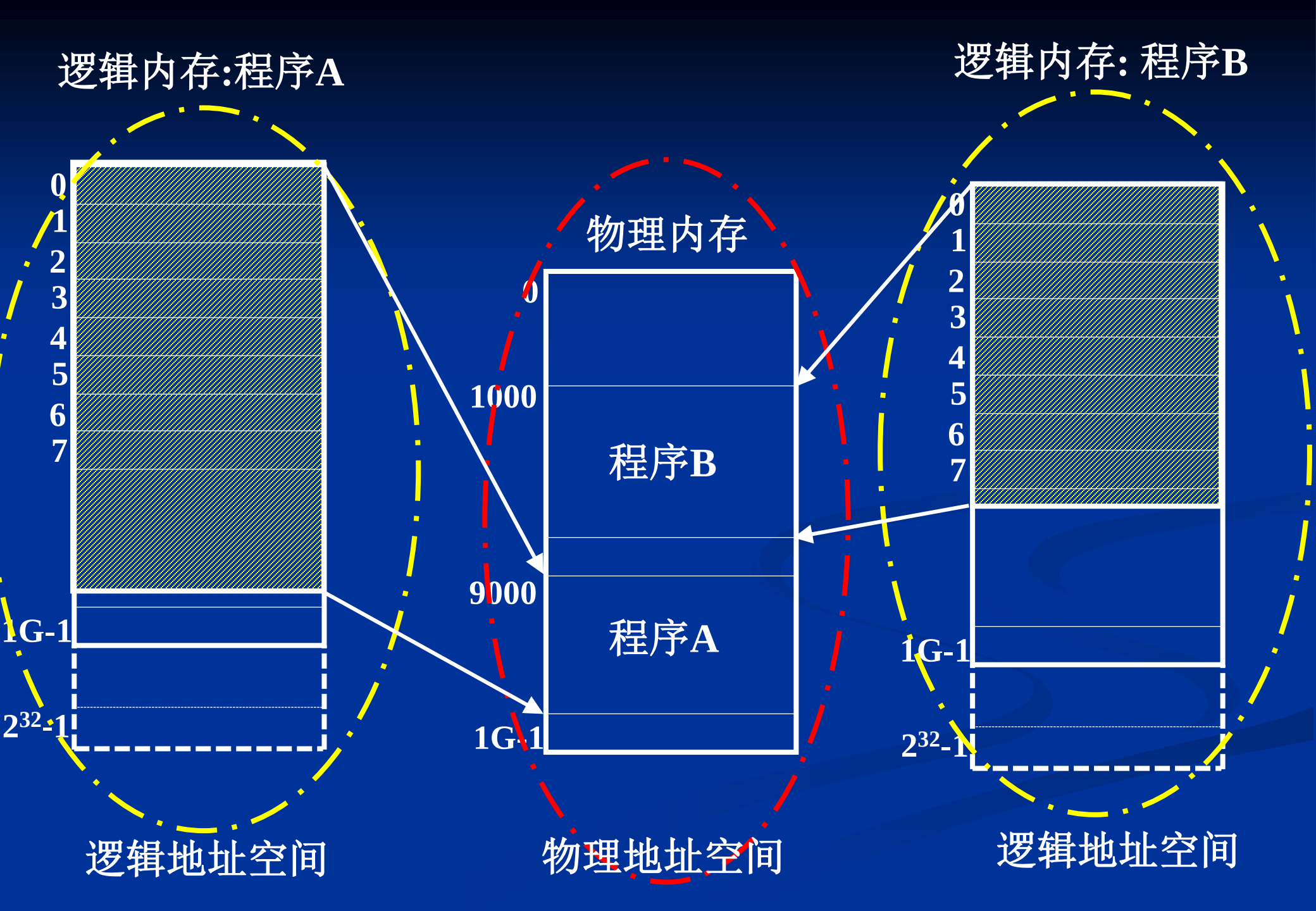
  装入时，进行地址重定位；

  特点：

  灵活，支持多道程序设计技术；

应用虚拟技术实现存储管理




逻辑内存:程序A
0
1
2
3
4
5
6
7
1G-1
2^{32}-1
逻辑地址空间
物理内存
0
1000
程序B
9000
程序A
1G-1
物理地址空间
逻辑内存: 程序B
0
1
2
3
4
5
6
7
1G-1
2^{32}-1
逻辑地址空间

应用虚拟技术实现存储管理

- 逻辑地址（相对地址，虚地址）

  用户程序经过汇编或编译后形成目标代码；

  目标代码采用相对地址形式:

  (1) 首地址为0;

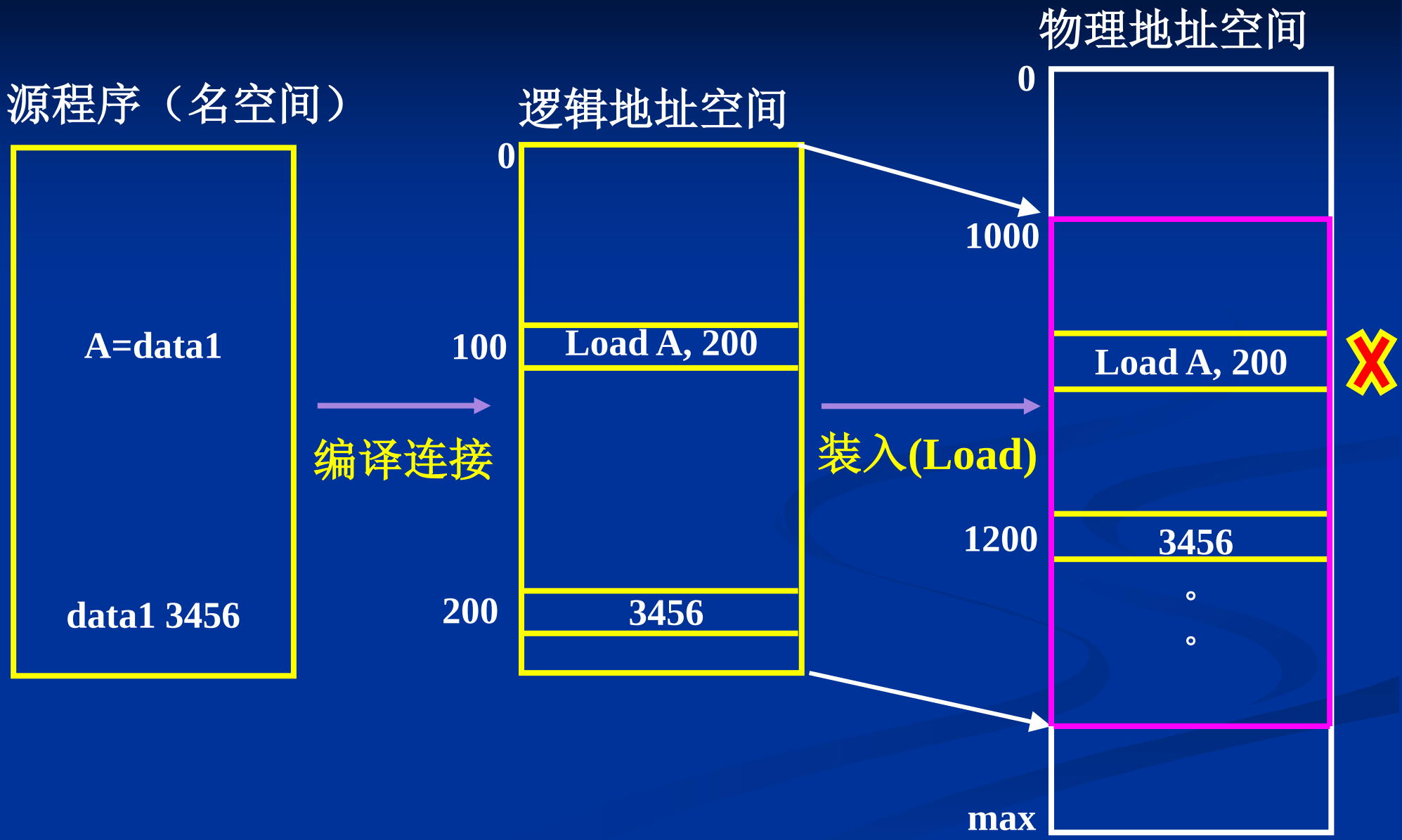
  (2) 其余指令中地址以相对于首地址的偏移为地址;

  不能用逻辑地址在内存中读取信息

- 物理地址（绝对地址，实地址）

  内存中存储单元的地址，可直接寻址

符号名空间、逻辑地址和物理地址



- 地址重定位（地址映射）

  为了保证CPU执行指令时可正确访问存储单元，需将用户程序中的逻辑地址转换为运行时由机器直接寻址的物理地址，这一过程称为地址映射。

  逻辑地址（相对地址，虚地址）

  物理地址（绝对地址，实地址）

- 静态地址重定位

  当用户程序被装入内存时，一次性实现逻辑地址到物理地址的转换，以后不再转换

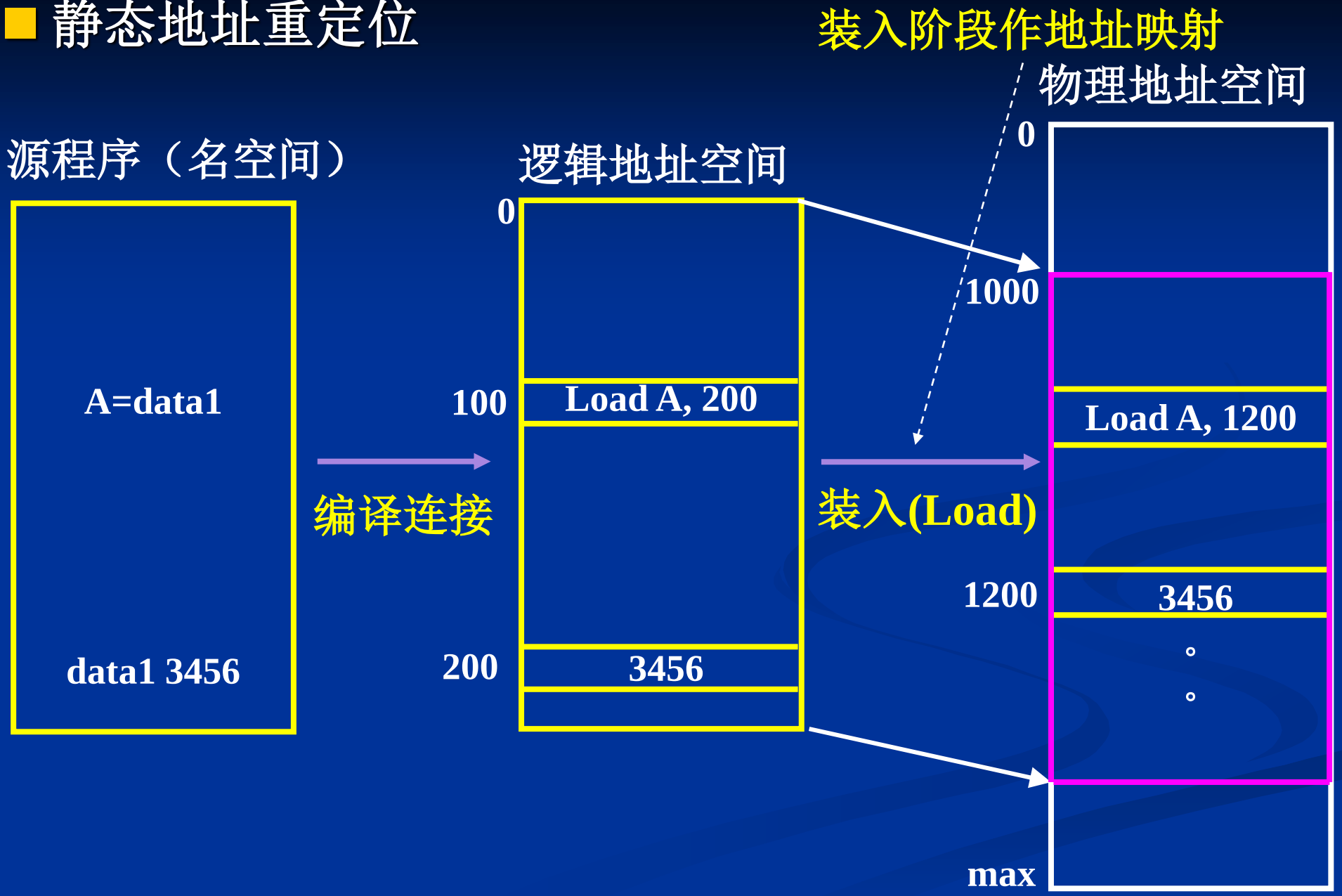
  一般在装入内存时由软件完成

- 动态地址重定位

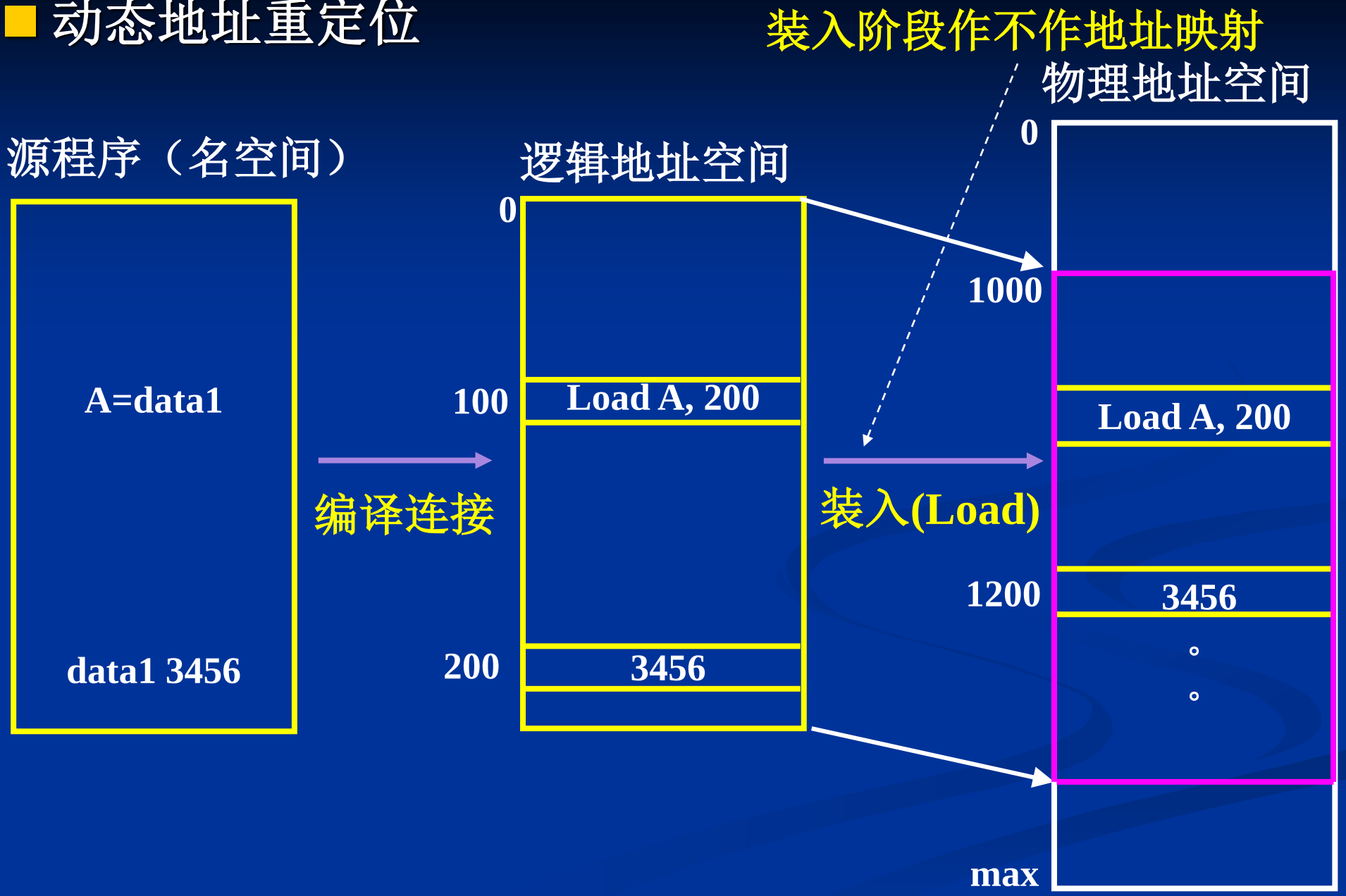
  在程序运行过程中要访问数据时再进行地址变换（即在逐条指令执行时完成地址映射。一般为了提高效率，此工作由硬件地址映射机制来完成。硬件支持，软硬件结合完成）

  硬件上需要一对寄存器的支持

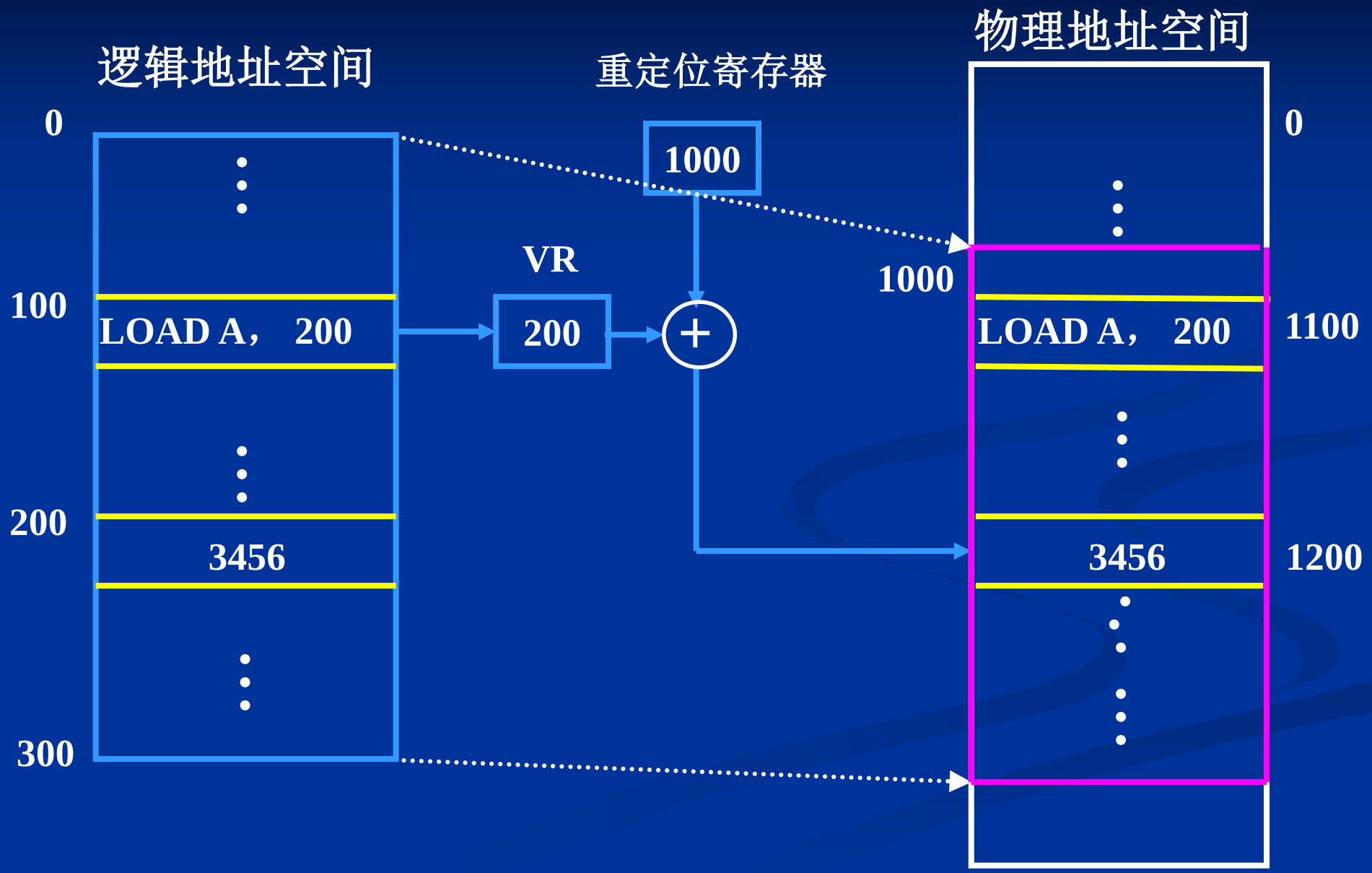
静态地址重定位


装入阶段作地址映射
物理地址空间
源程序（名空间）
逻辑地址空间
0
A=data1
100
Load A, 200
200
3456
data1 3456
编译连接
装入(Load)
0
1000
Load A, 1200
1200
3456
max

动态地址重定位


装入阶段作不作地址映射
物理地址空间
源程序（名空间）
逻辑地址空间
0
A=data1
data1 3456
100
Load A, 200
200
3456
编译连接
装入(Load)
0
1000
Load A, 200
1200
3456
max

动态地址重定位


逻辑地址空间
重定位寄存器
物理地址空间
0
100
200
300
1000
VR
200
+
1000
0
1100
1200
LOAD A，200
3456
LOAD A，200
3456

4.1 程序的装入和链接

4.1.1 程序的装入

- 动态运行时装入方式

  编程时，使用逻辑地址，运行时使用物理地址；

  运行时进行地址重定位；

  特点：

  灵活;支持多道程序设计技术；支持程序“搬家”

4.1.2 程序的链接

- 静态链接

  编程时，链接所有模块；

- 装入时动态链接

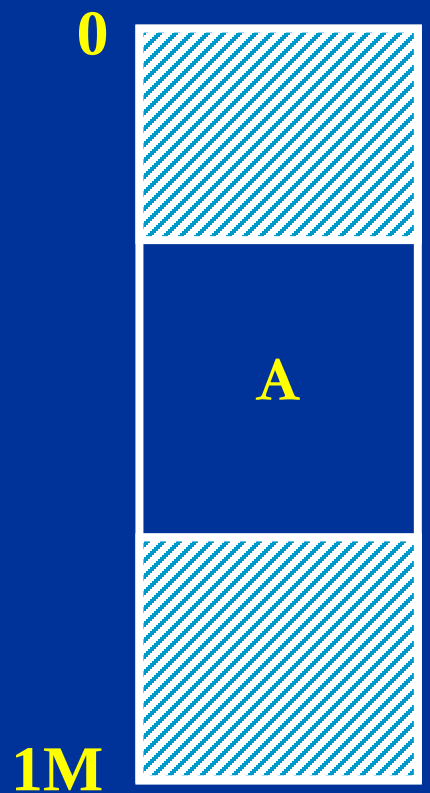
  装入时，链接所有模块；

- 运行时动态链接

  运行时，根据需要链接模块；

4.2 连续分配方式

- 连续分配方式
  为程序分配一段连续存储空间。



- 离散分配方式
  为程序分配若干段连续的存储空间。



4.2.1 单一连续分配存储管理方案

1. 应用背景

   单用户系统。

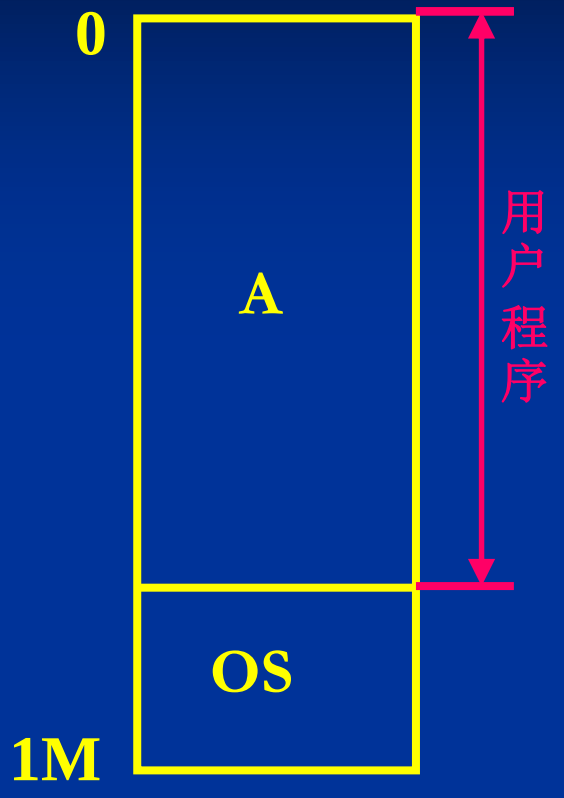
   在一段时间内，只有一个进程在内存。

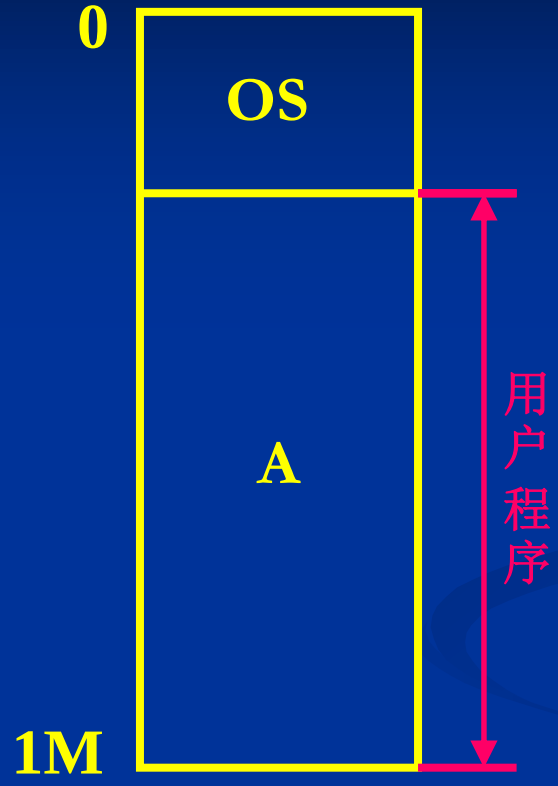
2. 基本思想

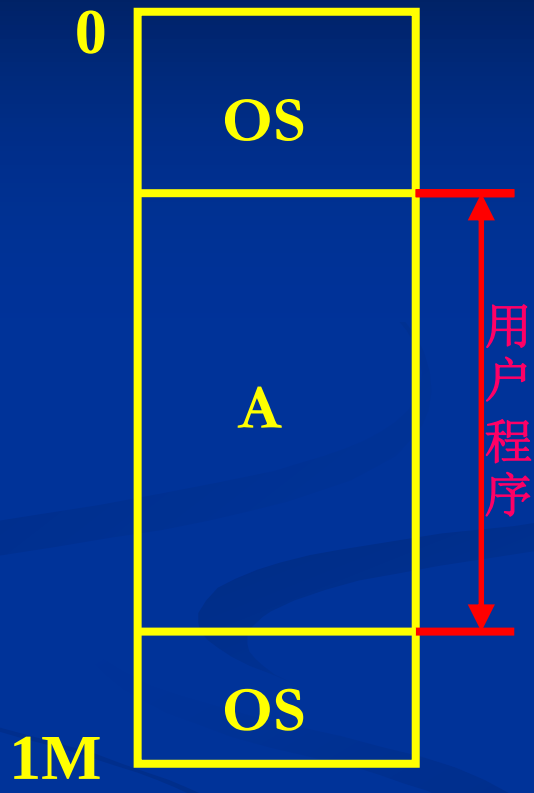
   内存分为两个区域：一个供操作系统使用，一个供用户使用



三种典型内存布局：



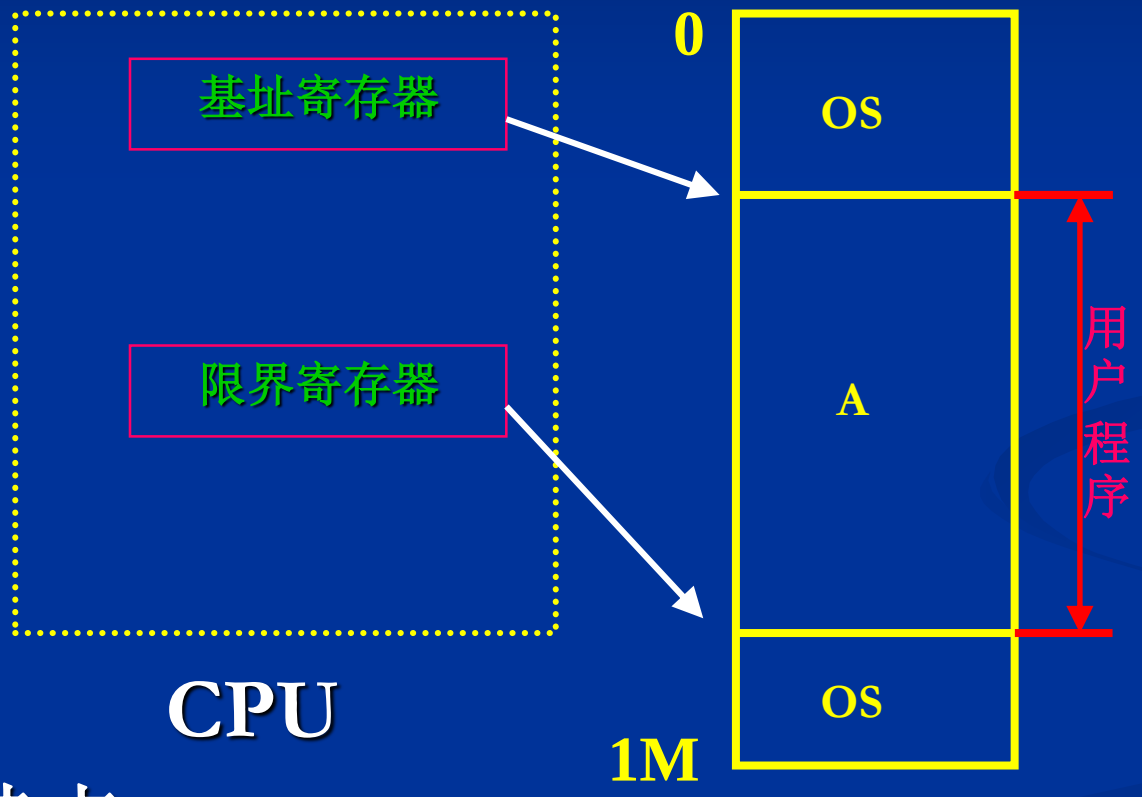




3. 实现

设置基址寄存器、限界寄存器，要求：

逻辑地址映射为物理地址后，物理地址必须在两者之间。



4. 特点

简单；适用于单任务OS；

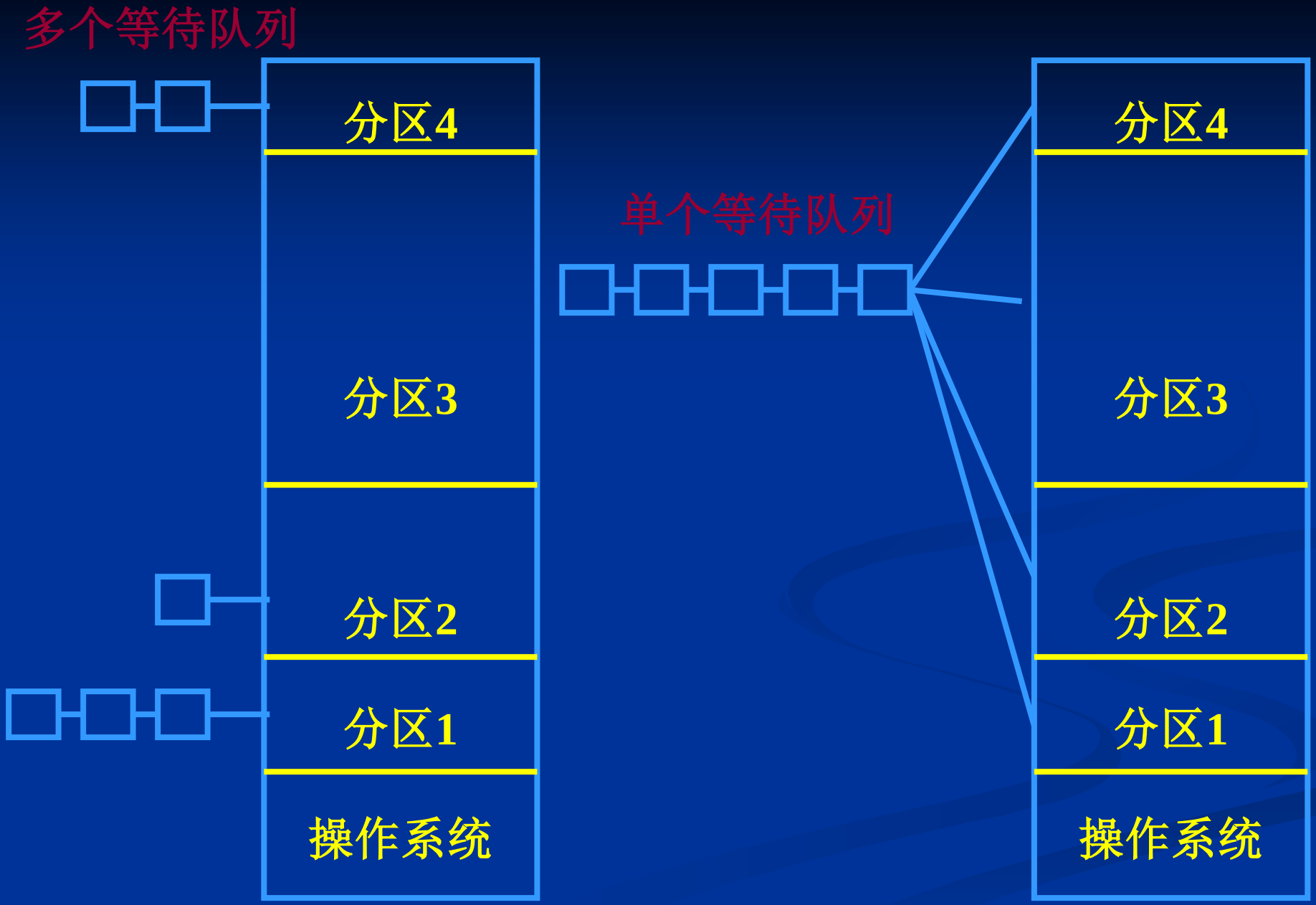
4.2.2 固定分区分配存储管理方案

1. 应用背景

   多道程序设计系统。

2. 基本思想

   (1) OS初始化阶段，将用户区划分为若干固定大小的内存区域（分区）；
   (2) 每个分区可放一个程序运行；
   (3) 一个分区内的程序执行结束，可调入另一个程序执行；
   (4) 划分为几个分区，则允许几个程序同时在内存运行；
   (5) 分区大小和位置可不同；

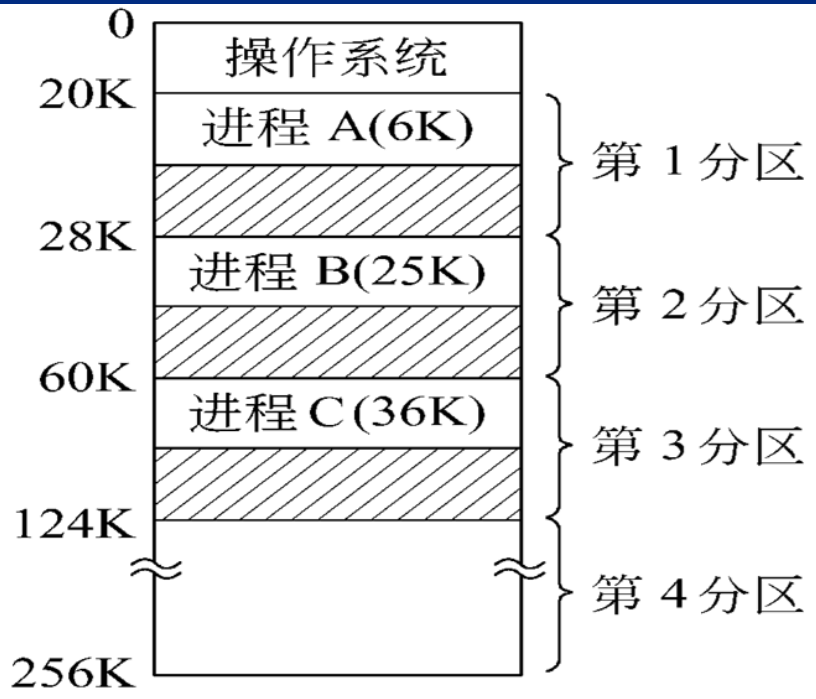

多个等待队列
分区4
分区3
分区2
分区1
操作系统
单个等待队列
分区4
分区3
分区2
分区1
操作系统

3. 实现

(1) 数据结构：分区说明表

区号	分区长度	起始地址	状态
1	8K	20K	已分配
2	32K	28K	已分配
3	64K	60K	已分配
4	132K	124K	未分配

(a) 分区说明表



(b) 内存状态

(2) 算法：分区分配算法；分区回收算法

4. 特点

(1)最早的支持多道程序设计的存储管理方案；

(2)简单，现在仍在嵌入式系统中使用；

(3)“内零头”严重；

内存的零头：

内零头：分配给进程，而进程未用到的内存部分；

外零头：未分配给进程，但因为太小而无进程能用；

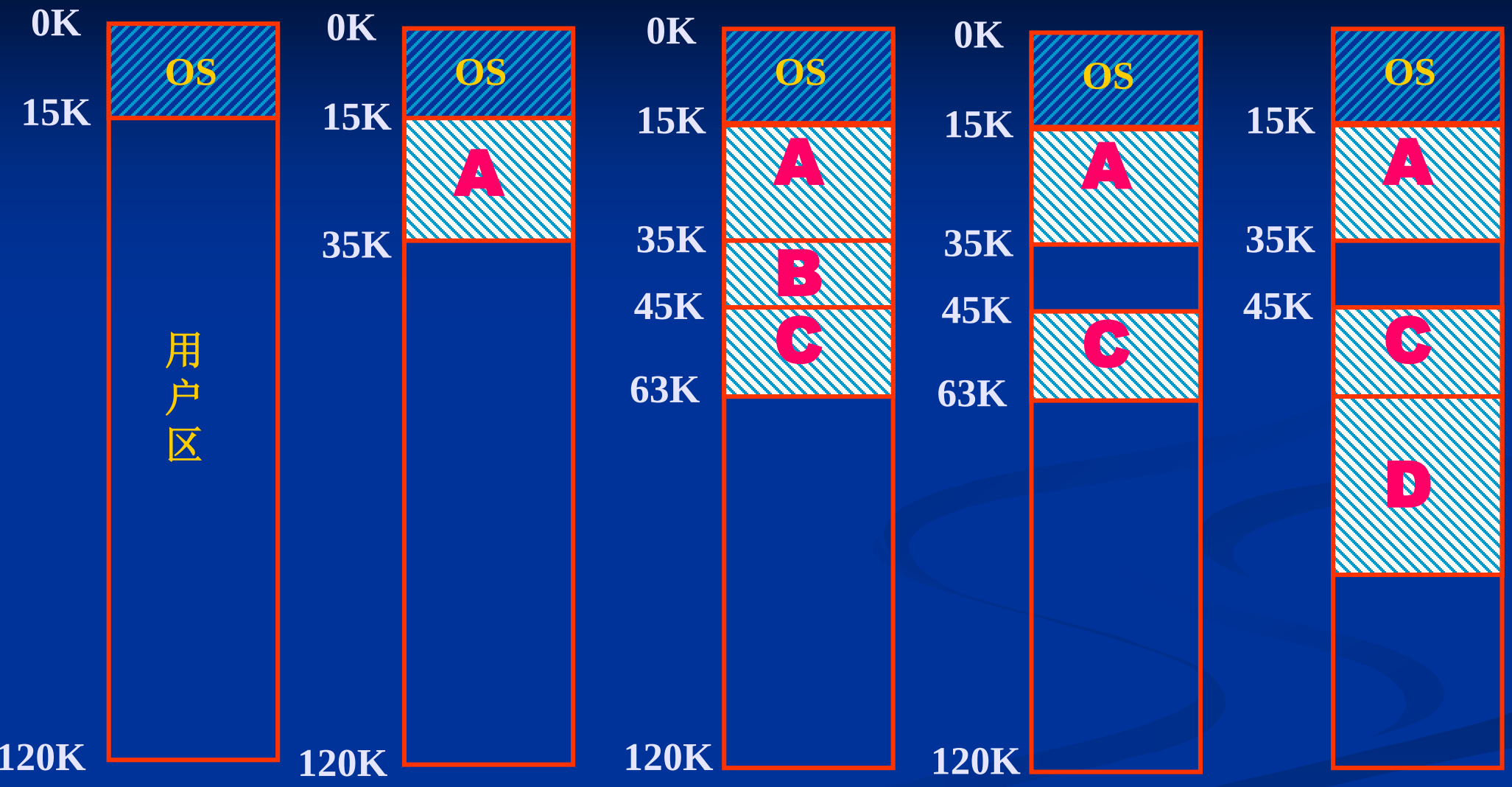
4.2.3 动态分区分配存储管理方案

1. 应用背景

   多道程序设计系统。

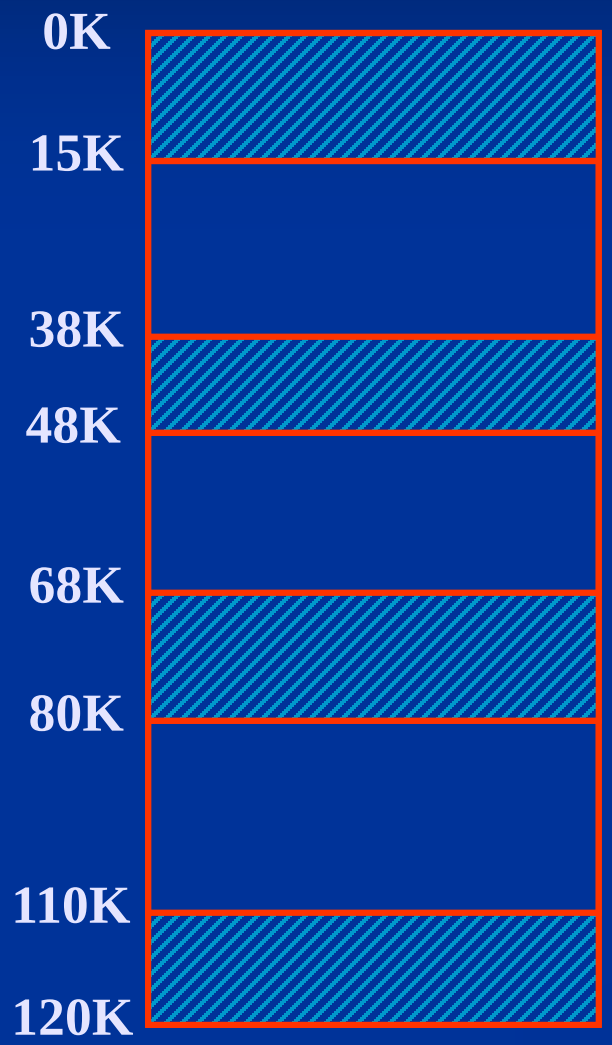
2. 基本思想

   (1) OS初始化阶段，将用户区划分为一个大的空白分区；
   (2) 每个程序运行时，划分出一个大小合适的分区；
   (3) 程序执行结束，其所在的分区撤消；


0K
OS
15K
用户区
120K
0K
OS
15K
A
35K
120K
0K
OS
15K
A
35K
B
45K
C
63K
120K
0K
OS
15K
A
35K
45K
C
63K
120K
0K
OS
15K
A
35K
45K
C
D

3. 实现

(1) 数据结构： (a)空闲分区表

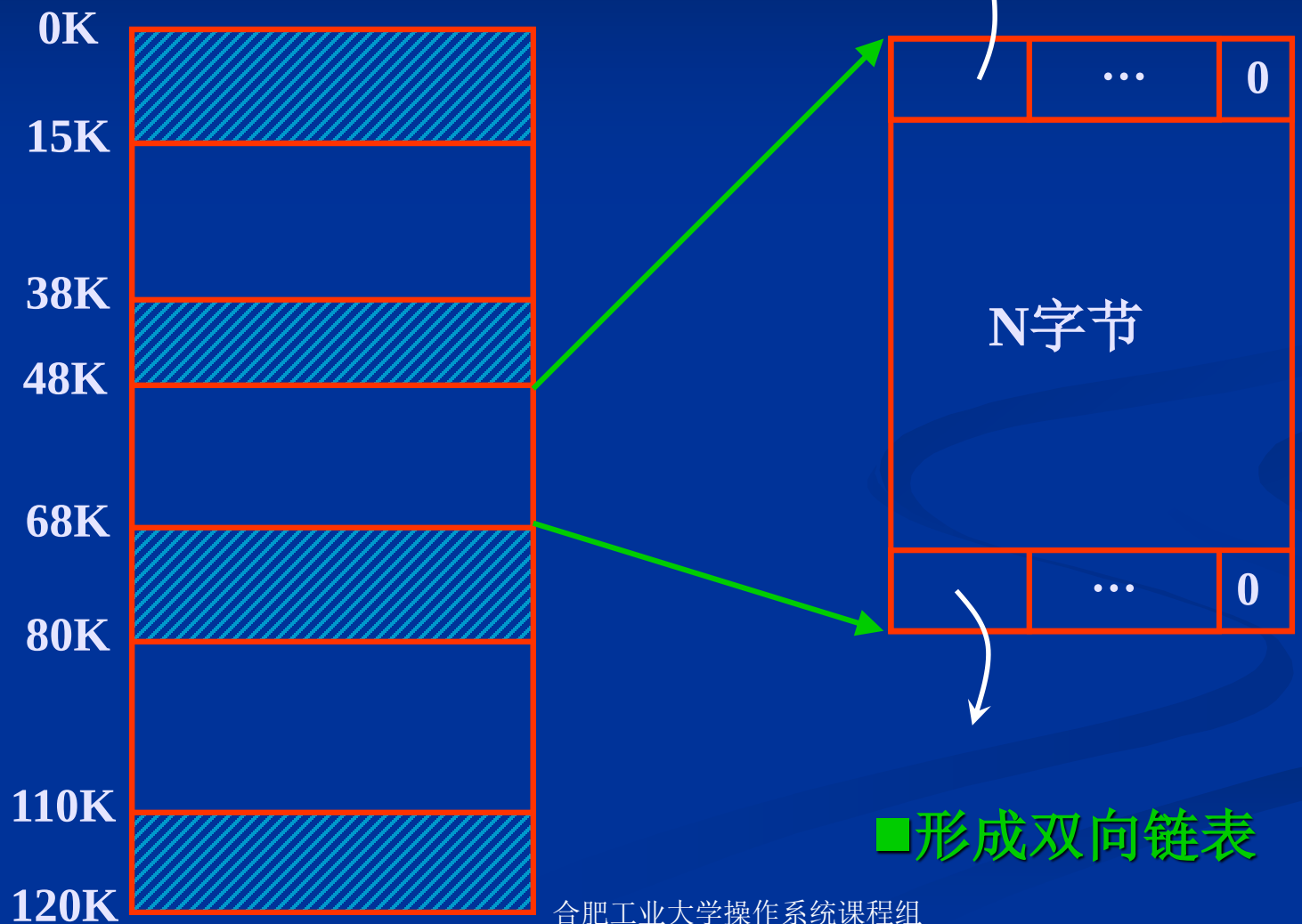


空闲分区表

始址	长度	标志
15K	23K	未分配
48K	20K	未分配
80K	30K	未分配
		空
		空

3. 实现

(1) 数据结构： (b)空闲分区链



■形成双向链表

3. 实现

(2) 算法：

(a) 分区分配算法

1) 首次适应算法

按地址递增顺序排列空闲分区链；

分配内存时，总是从低地址端开始扫描空闲区；

找到的第一个大小合适的分区，就分割分配；

否则失败。

特点：

高址端有大空闲区概率大；

低址端迅速被划分，碎片出现速度快；

2) 循环首次适应算法

按地址递增顺序排列空闲分区链；

设置当前指针；

分配内存时，从指针所指位置开始扫描空闲区；

找到的第一个大小合适的分区，就分割分配；

否则失败。

本质：

CLOCK算法；

特点：

碎片分布均匀；

高址端有大空闲区概率小；

3) 最佳适应算法

按大小递增顺序排列空闲分区链；

分配内存时，从链首开始扫描空闲区；

找到的第一个大小合适的分区，就分割分配；

否则失败。

特点：

碎片迅速出现；

“最佳”匹配；

4) 最坏适应算法

按大小递减顺序排列空闲分区链；

分配内存时，总是分配链首空闲区；

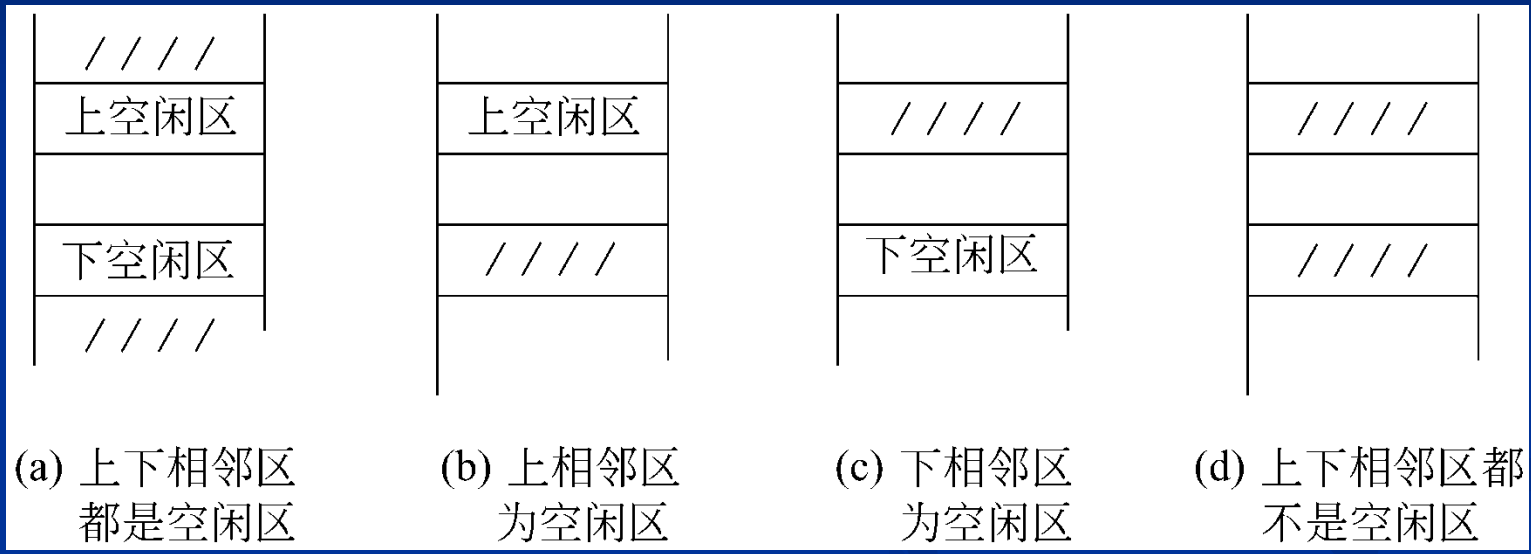
否则失败。

特点：

碎片出现最慢（分割后剩下的分区是最大的）；

“最坏”匹配；

(b) 分区回收算法



（a）和上下一起合并　　（c）和下一起合并

（b）和上一起合并　　（d）自己成为一个新区

4. 特点

- “碎片”问题

  经过一段时间的分配回收后，内存中存在很多很小的空闲块。它们每一个都很小，不足以满足分配要求；但其总和满足分配要求。这些空闲块被称为碎片造成存储资源的浪费

4.2.3 可重定位分区分配存储管理方案

1. 应用背景

   多道程序设计系统。

2. 基本思想

   (1) 采用动态分区分配方式;
   (2) 引入“紧凑”机制，合并小的碎片空闲区;
   (3) 使用动态地址重定位;



- 算法设计问题：如何紧凑，才能保证算法效率最高？